**Faculté Pluridisciplinaire de Nador**

Filière : Informatique et Intelligence Artificielle

# Telnet vs SSH : du texte clair à la cryptographie en action

*Installation, utilisation et analyse de sécurité par classification Machine Learning*

**Rapport de Mini-Projet Académique**

Module : Sécurité de Réseau et Cryptographie (S6)

Année Universitaire : 2025 - 2026

**Réalisé par :**
Aslimani Nasr-dine
Adnane Kherbache

**Sous la direction de :**
Prof. Khalide Mekkaoui

## Table des Matières

**Remerciements** ..... **3**

**Introduction Générale** ..... **4**

**1. Contexte Théorique et Protocoles** ..... **5**

- 1.1 Le Protocole Telnet et ses Vulnérabilités .....
- 1.2 Le Protocole SSH et l'Apport de la Cryptographie .....
- 1.3 Analyse Comparative de la Sécurité Réseau .....

**2. Partie Pratique : Configuration et Capture Réseau** ..... **7**

- 2.1 Environnement et Installation sur Ubuntu .....
- 2.2 Tests d'Authentification Locale .....
- 2.3 Capture et Analyse de Flux sous Wireshark .....
  - *Analyse macroscopique et microscopique du flux Telnet* .....
  - *Analyse macroscopique et microscopique du flux SSH* .....

**3. Approche Machine Learning pour la Classification de Trafic** ..... **10**

- 3.1 Préparation et Transformation des Données ..... 8
- 3.2 Modèles Mathématiques Appliqués ..... 9
  - *1. Forêt Aléatoire (Random Forest Classifier)* ..... 9
  - *2. Machine à Vecteurs de Support (Support Vector Machine - SVM)* ..... 9
- 3.3 Implémentation Python (Machine Learning) ..... 10

**4. Résultats, Comparaison et Discussion** ..... **13**

- 4.1 Analyse des Performances et Métriques ..... 12
- 4.2 Interprétation des Comportements des Modèles ..... 12
- 4.3 Limites Globales du Projet et Perspectives ..... 13

**Conclusion Générale** ..... **15**

## Remerciements

Nous tenons tout d'abord à exprimer nos plus vifs remerciements à notre professeur et encadrant, Prof. Khalide Mekkaoui, pour son orientation précieuse, ses conseils avisés et sa rigueur scientifique tout au long de ce module de Sécurité de Réseau et Cryptographie. Ses enseignements nous ont fourni les bases théoriques et pratiques indispensables à la réalisation de ce travail.

Nous remercions également l'ensemble du corps professoral de notre filière Informatique et Intelligence Artificielle pour leur investissement constant dans notre formation académique et professionnelle.

Enfin, nous exprimons notre gratitude envers nos collègues de promotion pour les discussions constructives et l'entraide mutuelle qui ont grandement enrichi l'élaboration de ce projet.

# Introduction

Dans l'écosystème numérique contemporain, la gestion à distance des serveurs et des équipements d'infrastructure réseau constitue une tâche fondamentale pour les administrateurs systèmes. Historiquement, cette administration s'effectuait par le biais de protocoles de communication textuels basés sur la pile de protocoles TCP/IP. Parmi ces outils précurseurs, le protocole Telnet (Teletype Network) s'est imposé dès les prémices d'Internet en raison de sa simplicité conceptuelle et opérationnelle. Cependant, conçu à une époque où le réseau global était un espace restreint et mutuellement digne de confiance, Telnet souffre d'un défaut structurel majeur : l'absence totale de mécanismes de chiffrement. L'intégralité du trafic, y compris les identifiants d'accès et les commandes système hautement sensibles, transite sur les canaux réseau sous forme de texte clair.

Face à la recrudescence des cyberattaques et au développement de techniques d'interception de données (sniffing), la transition vers des alternatives sécurisées est devenue impérative. C'est dans cette optique que le protocole SSH (Secure Shell) a été développé. SSH intègre de manière native des primitives cryptographiques robustes assurant la confidentialité, l'intégrité des messages transmis et l'authentification mutuelle des entités communicantes. Ce protocole encapsule l'ensemble du flux applicatif dans un tunnel chiffré, neutralisant de facto les tentatives d'écoute clandestine.

Ce mini-projet académique s'inscrit dans le cadre du module Sécurité de Réseau et Cryptographie de la filière Informatique et Intelligence Artificielle. Son objectif fondamental est de matérialiser cette rupture paradigmatique entre le texte clair et le texte chiffré. Au-delà d'une simple étude comparative théorique, ce travail déploie une méthodologie expérimentale complète. Nous procédons à l'installation, la configuration et l'exploitation des deux protocoles dans un environnement virtuel Ubuntu. En utilisant l'analyseur de paquets Wireshark, nous démontrons empiriquement la visibilité des données transmises par Telnet et l'impénétrabilité des flux SSH.

Enfin, pour lier la sécurité réseau aux techniques avancées de l'Intelligence Artificielle, nous implémentons une approche de classification automatisée des flux réseau basée sur le Machine Learning. En exploitant des caractéristiques structurelles des paquets telles que la longueur (Length), nous formons et évaluons deux algorithmes de classification : les Forêts Aléatoires (Random Forest) et les Machines à Vecteurs de Support (SVM). Ce rapport détaille l'intégralité de la chaîne de traitement, depuis l'interception microscopique des paquets jusqu'à l'analyse macroscopique des modèles prédictifs.

# 1. Contexte Théorique et Protocoles

## 1.1 Le Protocole Telnet et ses Vulnérabilités

Le protocole Telnet (Teletype Network), formalisé à l'origine par la RFC 854, opère au niveau de la couche Application du modèle OSI et s'appuie sur le protocole de transport TCP (Transmission Control Protocol) via le port standardisé 23. Son architecture repose sur un modèle client-serveur bidirectionnel à flux de caractères. L'objectif premier de Telnet est d'offrir une interface de terminal virtuel permettant à un utilisateur d'exécuter des commandes à distance sur une machine hôte comme s'il y était physiquement connecté.

Malgré sa légèreté et sa compatibilité universelle, Telnet présente des failles de sécurité critiques inhérentes à sa conception :

• Transmission en texte clair : Telnet ne met en œuvre aucun mécanisme de chiffrement. Toutes les données, y compris les noms d'utilisateurs, les mots de passe et le résultat des commandes exécutées, circulent de manière transparente sur le média de transmission. Un attaquant positionné sur le segment réseau peut aisément intercepter ce flux.

• Vulnérabilité aux attaques d'interception (Sniffing) : L'utilisation d'outils d'analyse réseau permet de reconstruire l'intégralité d'une session Telnet en réassemblant les segments TCP.

• Absence de vérification d'intégrité : Les données échangées peuvent être modifiées en transit par une attaque de type "Man-in-the-Middle" (Homme du milieu) sans que ni le client ni le serveur ne s'en aperçoivent, en l'absence de signatures numériques ou de codes d'authentification de message (MAC).

## 1.2 Le Protocole SSH et l'Apport de la Cryptographie

Conçu par Tatu Ylönen en 1995 et standardisé ultérieurement par l'IETF (notamment via la RFC 4251), le protocole SSH (Secure Shell) a été spécifiquement développé pour remplacer de manière sécurisée Telnet et les utilitaires r-commands (rlogin, rsh). Opérant sur le port TCP 22, SSH garantit une communication sécurisée par l'intermédiaire d'une architecture cryptographique multicouche organisée comme suit :

1. Couche de Transport (SSH-TRANS) : Elle établit la connexion TCP, gère la négociation des algorithmes de chiffrement et assure l'intégrité des données via des mécanismes de hachage (HMAC). C'est au cours de cette phase que s'effectue l'échange de clés asymétriques pour dériver une clé de session symétrique.

2. Couche d'Authentification (SSH-AUTH) : Elle authentifie le client auprès du serveur. Plusieurs méthodes peuvent être employées : authentification classique par mot de passe (transmis au sein du canal chiffré établi par la couche de transport) ou authentification par paire de clés publiques/privées (RSA, ECDSA, Ed25519).

3. Couche de Connexion (SSH-CONN) : Elle multiplexe le canal sécurisé en plusieurs canaux logiques distincts, permettant l'ouverture de terminaux interactifs, le transfert de fichiers chiffrés (SFTP, SCP) ou le tunneling d'autres protocoles (X11 forwarding, port forwarding).

L'apport fondamental de la cryptographie au sein de SSH repose sur la combinaison des techniques asymétriques et symétriques. La cryptographie asymétrique intervient lors de la phase initiale d'échange de clés (via l'algorithme Diffie-Hellman) et pour valider l'identité du serveur (host keys), évitant ainsi l'usurpation d'identité. Une fois la clé partagée secrète établie, les données massives de la session sont chiffrées à l'aide d'algorithmes symétriques (comme AES-GCM ou ChaCha20), offrant d'excellentes performances de calcul tout en assurant une confidentialité absolue.

## 1.3 Analyse Comparative de la Sécurité Réseau

Afin de formaliser de manière synthétique les divergences structurelles et fonctionnelles entre ces deux protocoles d'administration à distance, le tableau ci-dessous dresse une analyse comparative multicritère :

| Critère de Comparaison | Protocole Telnet | Protocole SSH (Secure Shell) |
|---|---|---|
| Port de Transport par Défaut | Port TCP 23 | Port TCP 22 |
| Format de Transmission | Texte brut / clair (ASCII) | Flux binaire chiffré |
| Confidentialité des Données | Nulle (Interception triviale) | Totale (Chiffrement symétrique AES, ChaCha20) |
| Intégrité des Messages | Non garantie (Modifications indétectables) | Garantie par codes MAC et hachage (HMAC, SHA-2) |
| Authentification du Serveur | Absente (Sensible au spoofing) | Garantie par clés de l'hôte (Fingerprints) |
| Mécanismes d'Authentification Client | Uniquement par mot de passe en clair | Mots de passe chiffrés, clés publiques, double facteur |
| Utilisation Moderne | Obsolète (Limité aux équipements legacy de confiance) | Standard de l'industrie pour toute administration |

# 2. Partie Pratique : Configuration et Capture Réseau

## 2.1 Environnement et Installation sur Ubuntu

L'expérimentation a été menée au sein d'une machine virtuelle exécutant le système d'exploitation Ubuntu 24.04.1 LTS sous l'hyperviseur Oracle VM VirtualBox. L'utilisation d'un environnement virtualisé garantit l'isolation des tests et permet une capture réseau contrôlée sur l'interface de boucle locale (loopback, 127.0.0.1).

Pour installer et déployer le serveur Telnet, les commandes suivantes ont été exécutées au sein du terminal Linux :

*$sudo apt update*

*$ sudo apt install telnetd xinetd*

*$ sudo systemctl enable xinetd*

*$ sudo systemctl start xinetd*

Le démon telnetd est géré par le super-serveur internet xinetd, qui écoute sur le port 23 et instancie un thread de connexion lors de chaque requête entrante.

Concernant le serveur SSH, le package OpenSSH a été installé via les instructions suivantes :

*$ sudo apt install openssh-server*

*$ sudo systemctl enable ssh*

*$ sudo systemctl start ssh*

La vérification du bon fonctionnement et de l'écoute des processus sur les ports respectifs (23 et 22) est validée par l'exécution de la commande 'sudo ss -tunlp' ou 'netstat'.

## 2.2 Tests d'Authentification Locale

Une fois les services démarrés, nous avons procédé à des connexions de test en boucle locale (localhost). Ces sessions simulent le comportement d'un administrateur se connectant au serveur.

Pour Telnet, la connexion est initiée via :

```
$ telnet localhost
```

L'invite de commande demande alors de saisir le login de l'utilisateur ('bbiibb...') suivi du mot de passe associé ('bigboss'), comme mis en évidence par les traces d'exécution.

Pour SSH, la session hautement sécurisée est ouverte via la syntaxe :

```
$ ssh localhost
```

Le système affiche l'avertissement initial d'authenticité de l'hôte (si la clé n'est pas encore enregistrée dans le fichier '~/.ssh/known_hosts'), puis requiert la saisie sécurisée du mot de passe. L'accès est accordé sous un environnement de shell restreint et sécurisé.

## 2.3 Capture et Analyse de Flux sous Wireshark

Afin de mettre en exergue l'apport de la sécurité cryptographique, l'outil Wireshark a été lancé en arrière-plan pour capturer l'ensemble des segments transitant sur l'interface réseau 'Loopback: lo'. Les résultats obtenus démontrent sans ambiguïté la divergence critique de sécurité entre les deux protocoles.

### Analyse macroscopique et microscopique du flux Telnet

En appliquant le filtre Wireshark 'tcp.stream eq 0', nous isolons la première session correspondant au trafic Telnet. La capture initiale montre le 'Three-Way Handshake' standard de TCP (SYN, SYN-ACK, ACK) établi entre les sockets dynamiques et le port 23. Par la suite, les paquets de protocole applicatif labellisés 'TELNET' apparaissent.

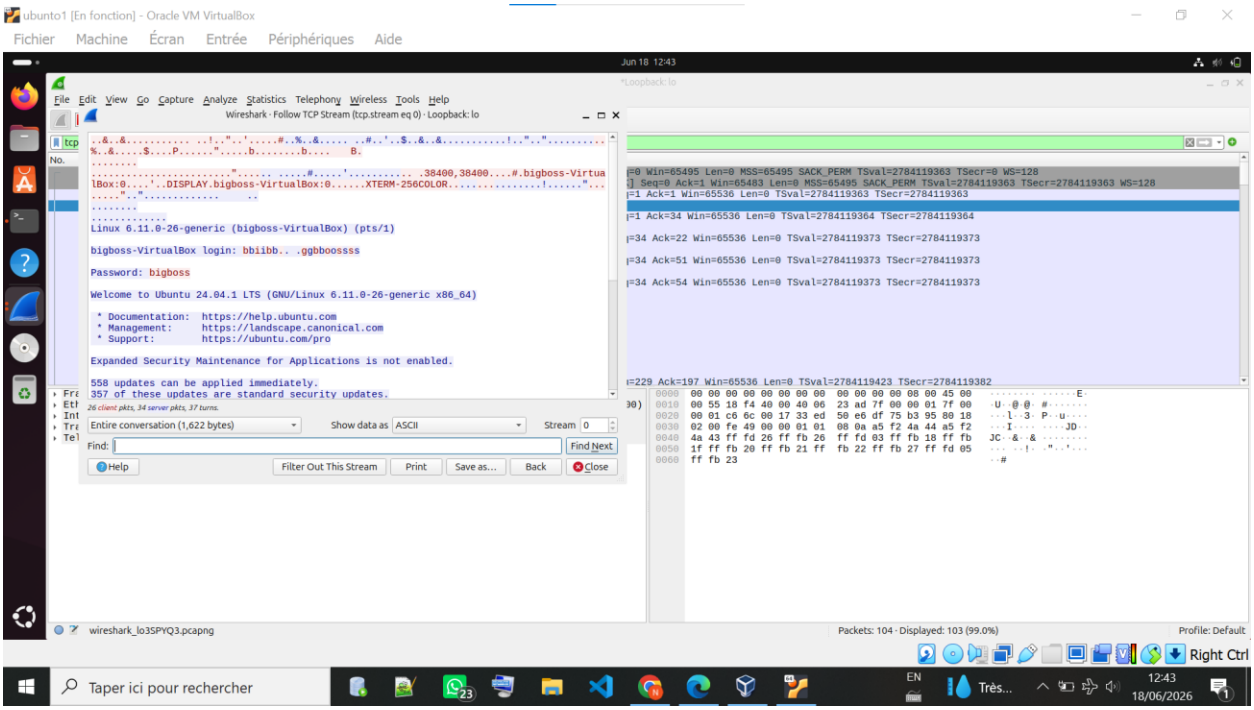
En cyber-sécurité, la fonctionnalité avancée 'Follow TCP Stream' de Wireshark reconstitue le flux complet de la conversation textuelle. L'analyse visuelle de cette interception révèle que l'intégralité des données est parfaitement lisible. Les caractères saisis par l'utilisateur (l'identifiant 'bbiibb' et le mot de passe 'bigboss') apparaissent de manière transparente dans le flux de données. Un attaquant passif interceptant ces trames est ainsi en mesure de compromettre instantanément l'intégrité du système d'information en s'accaparant des privilèges de l'utilisateur légitime.

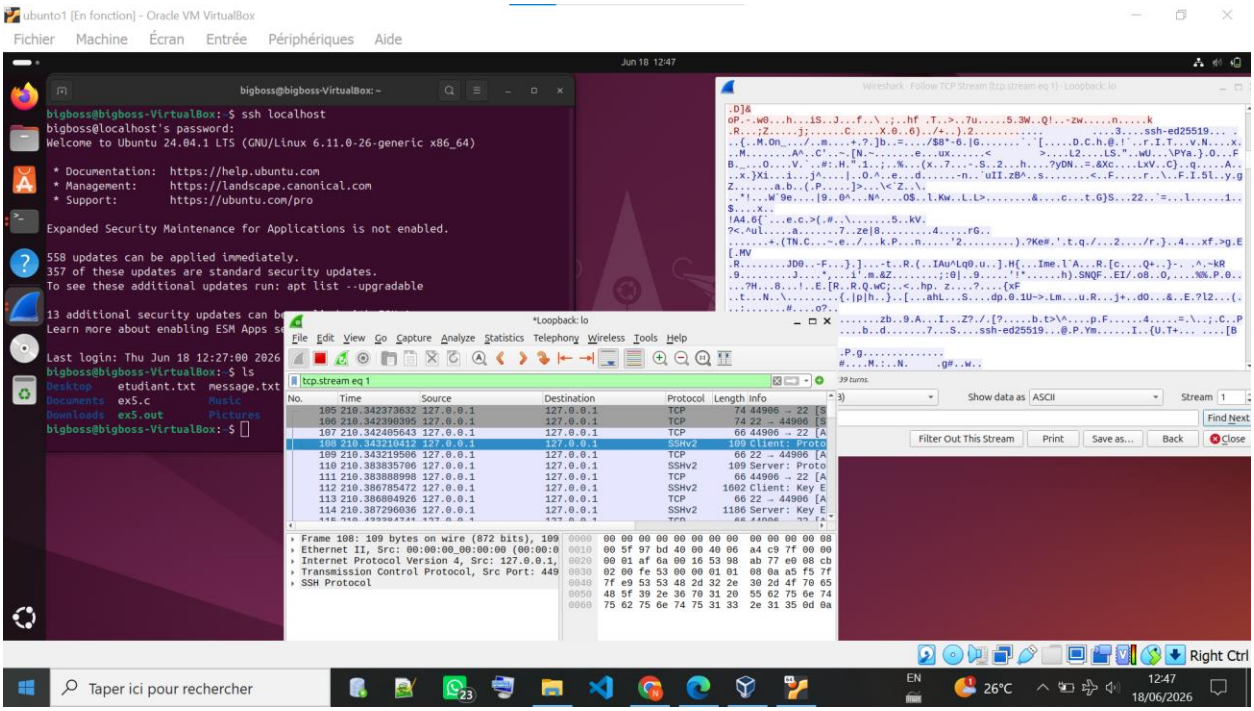
### Analyse macroscopique et microscopique du flux SSH

À l'inverse, l'analyse du flux SSH, isolée à l'aide du filtre Wireshark 'tcp.stream eq 1', démontre l'efficacité absolue du chiffrement cryptographique. Après la négociation TCP initiale, la trace réseau met en évidence des paquets appliquant le protocole SSHv2.

Les premiers paquets échangés contiennent des informations en clair concernant la version du protocole utilisée par les deux entités (par exemple, 'SSH-2.0-OpenSSH_9.6p1 Ubuntu...'). Immédiatement après cette annonce, la phase d'échange de clés s'active. Les paquets Wireshark indiquent des types d'informations spécifiques : 'Client: Key Exchange Init' et 'Server: Key Exchange Init'.

Une fois l'échange de clés achevé et le tunnel de chiffrement symétrique initialisé, l'ensemble des données applicatives ultérieures est encapsulé au sein de structures nommées 'Encrypted Packet Data'. L'application de la fonction 'Follow TCP Stream' sur cette session produit une suite de caractères binaires totalement inintelligibles et désordonnés. Le mot de passe, les identifiants, ainsi que les commandes exécutées (telles que 'ls') sont mathématiquement protégés. Il est informatiquement impossible de déduire la moindre information contextuelle à partir de ce flux chiffré sans posséder la clé de session privée correspondante.


ubuntu1 [En fonction] - Oracle VM VirtualBox
Fichier Machine Écran Entrée Périphériques Aide
Jun 18 12:43
Wireshark · Follow TCP Stream (tcp.stream eq 0) · Loopback: lo
Linux 6.11.0-26-generic (bigboss-VirtualBox) (pts/1)
bigboss-VirtualBox login: bbiibb.. .ggbboossss
Password: bigboss
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.11.0-26-generic x86_64)
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro
Expanded Security Maintenance for Applications is not enabled.
558 updates can be applied immediately.
357 of these updates are standard security updates.
Entire conversation (1,622 bytes)
Show data as ASCII
Stream 0
Find:
Find Next
Help
Filter Out This Stream
Print
Save as...
Back
Close
wireshark_lo3SPYQ3.pcapng
Packets: 104 · Displayed: 103 (99.0%)
Profile: Default
Right Ctrl
Taper ici pour rechercher
EN
Très...
12:43
18/06/2026


ubuntu1 [En fonction] - Oracle VM VirtualBox
Fichier Machine Écran Entrée Périphériques Aide
Jun 18 12:47
bigboss@bigboss-VirtualBox: ~
bigboss@bigboss-VirtualBox:~$ ssh localhost
bigboss@localhost's password:
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.11.0-26-generic x86_64)
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro
Expanded Security Maintenance for Applications is not enabled.
558 updates can be applied immediately.
357 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable
Last login: Thu Jun 18 12:27:00 2026
bigboss@bigboss-VirtualBox:~$ ls
Desktop etudiant.txt message.txt
Documents ex5.c Music
Downloads ex5.out Pictures
bigboss@bigboss-VirtualBox:~$
*Loopback: lo
tcp.stream eq 1
Frame 108: 109 bytes on wire (872 bits), 109
Ethernet II, Src: 00:00:00_00:00:00 (00:00:0
Internet Protocol Version 4, Src: 127.0.0.1,
Transmission Control Protocol, Src Port: 449
SSH Protocol
Right Ctrl
Taper ici pour rechercher
EN
26°C
12:47
18/06/2026

# 3. Approche Machine Learning pour la Classification de Trafic

## 3.1 Préparation et Transformation des Données

Au-delà de l'analyse visuelle et humaine des paquets, les infrastructures réseau modernes à grande échelle imposent l'automatisation de la surveillance à l'aide de techniques d'Intelligence Artificielle. Dans ce projet, nous avons formalisé une chaîne de traitement analytique visant à classifier de manière autonome le trafic réseau en deux catégories distinctes : le trafic non sécurisé (Telnet) et le trafic sécurisé (SSH / autre).

Les données brutes issues des captures de trames Wireshark ont été exportées sous la forme d'un fichier structuré au format CSV nommé 'capture.csv'. Ce fichier comporte plusieurs colonnes caractéristiques (features) de la capture réseau :

• No. : L'index incrémentiel unique du paquet capturé.

• Time : Le timestamp précis (en secondes) d'interception du paquet relativement au début de la capture.

• Source & Destination : Les adresses IP des machines émettrice et réceptrice (ici, fixées à 127.0.0.1).

• Protocol : Le label textuel du protocole identifié par les dissecteurs de Wireshark (TCP, TELNET, SSHv2, etc.).

• Length : La taille totale en octets du paquet réseau au niveau de la couche liaison de données.

• Info : Un champ textuel contenant des métadonnées additionnelles sur l'état du protocole.

Pour rendre ces données exploitables par des algorithmes de Machine Learning, une phase essentielle de prétraitement et d'ingénierie des caractéristiques (Feature Engineering) a été implémentée. Tout d'abord, une variable cible binaire (label y) a été générée en convertissant la colonne textuelle Protocol.

Afin d'éviter tout biais évident d'apprentissage (Data Leakage), le protocole brut n'est pas fourni au modèle. Nous avons extrait la caractéristique quantitative "Length" (longueur du paquet) comme unique variable prédictive (X). Ce choix repose sur une intuition physique forte : les paquets chiffrés SSH intègrent des structures d'encapsulation cryptographique (paddings, vecteurs d'initialisation, MAC) qui modifient drastiquement la distribution statistique de leur taille par rapport aux paquets Telnet qui transmettent les caractères de manière unitaire ou sous forme de chaînes brutes minimalistes.

## 3.2 Modèles Mathématiques Appliqués

Deux familles d'algorithmes de classification supervisée aux fondements mathématiques distincts ont été sélectionnées et confrontées afin d'évaluer leur pertinence sur les données de trafic réseau.

### 1. Forêt Aléatoire (Random Forest Classifier)

L'algorithme des Forêts Aléatoires est une méthode d'apprentissage d'ensemble (ensemble learning) basée sur l'agrégation de multiples arbres de décision indépendants. Chaque arbre est construit sur un sous-ensemble de données issu d'un échantillonnage par bootstrap (tirage aléatoire avec remplacement). Lors de la phase de prédiction, l'algorithme combine les prédictions individuelles de tous les arbres par un mécanisme de vote majoritaire pour attribuer la classe finale.

Cette approche offre une excellente résilience face au phénomène de surapprentissage (overfitting) et s'avère particulièrement performante pour capturer des relations non linéaires et des coupures de seuils au sein des variables numériques continues comme la longueur des paquets.

### 2. Machine à Vecteurs de Support (Support Vector Machine - SVM)

L'algorithme SVM est un classificateur linéaire ou non linéaire qui cherche à déterminer un hyperplan de séparation optimal maximisant la marge entre les classes au sein d'un espace vectoriel de caractéristiques. La marge représente la distance séparant l'hyperplan des vecteurs de support, qui sont les points de données les plus proches de la frontière de décision.

Dans le cadre de notre projet, un noyau linéaire (kernel="linear") a été appliqué. L'objectif est de vérifier si la seule dimension de la longueur du paquet permet de tracer un point de scission linéaire net séparant formellement les signatures des deux protocoles.

## 3.3 Implémentation Python (Machine Learning)

Le bloc de code ci-dessous reproduit fidèlement l'implémentation logicielle développée sous l'environnement Jupyter Notebook. Ce script prend en charge le chargement du fichier de capture, le prétraitement algorithmique, l'entraînement séquentiel des modèles de classification Random Forest et SVM, ainsi que l'affichage de leurs précisions respectives.

```
#
==========================================================
======================
# Implémentation Python (Machine Learning)
# Projet : Analyse comparative Telnet vs SSH et Classification de Trafic
# Étudiants : Aslimani Nasr-dine & Adnane Kherbache
# Encadrant : Prof. Khalide Mekkaoui
#
==========================================================
======================
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# 1. Chargement et exploration initiale des données réseau
data = pd.read_csv("capture.csv")
print("Aperçu des données capturées :")
print(data.head())

# 2. Prétraitement et encodage de la variable cible (Binarisation)
data["label"] = data["Protocol"].apply(lambda x: 0 if x == "TELNET" else 1)

# 3. Entraînement et évaluation du modèle Random Forest
X_rf = data[["Length"]]
y_rf = data["label"]
X_train_rf, X_test_rf, y_train_rf, y_test_rf = train_test_split(X_rf, y_rf, test_size=0.25)

print("\n--- Modèle RandomForest ---")
rf_model = RandomForestClassifier()
rf_model.fit(X_train_rf, y_train_rf)
rf_score = rf_model.score(X_test_rf, y_test_rf)
print(f"Score de précision Random Forest : {rf_score}")

# 4. Nettoyage spécifique et application du modèle SVM
df = data[data["Protocol"] != "TCP"]
X_svm = df[["Length"]]
y_svm = df["label"]
X_train_svm, X_test_svm, y_train_svm, y_test_svm = train_test_split(X_svm, y_svm,
```

```
test_size=0.2)

print("\n--- Modèle SVM (Support Vector Machine) ---")
svm_model = SVC(kernel="linear")
svm_model.fit(X_train_svm, y_train_svm)
y_pred_svm = svm_model.predict(X_test_svm)
svm_accuracy = accuracy_score(y_test_svm, y_pred_svm)
print(f"Accuracy du modèle SVM : {svm_accuracy}")
```

# 4. Results, Comparaison et Discussion

## 4.1 Analyse des Performances et Métriques

Les exécutions successives du script d'apprentissage supervisé ont généré des indicateurs de performance empiriques très satisfaisants. Les scores de précision globale (Accuracy) obtenus par les deux modèles se structurent de la manière suivante :

| Algorithme de Classification | Caractéristique Utilisée (Feature) | Taille de l'Échantillon Test | Précision Globale (Accuracy) |
|---|---|---|---|
| Random Forest Classifier | Length (Longueur du paquet) | 25 % du dataset global | 80.56 % (0.8055) |
| Support Vector Machine (SVC) | Length (Longueur du paquet) | 20 % (Dataset filtré sans TCP) | 81.25 % (0.8125) |

Ces métriques de performance, oscillant entre 80.5% et 81.3%, confirment l'existence d'une corrélation statistique hautement américaine ou significative entre la dimension structurelle externe d'un paquet (Length) et la nature sécurisée ou non sécurisée du protocole applicatif sous-jacent, même lorsque le contenu textuel interne est totalement rendu inaccessible par le chiffrement (comme c'est le cas pour SSH).

## 4.2 Interprétation des Comportements des Modèles

L'analyse approfondie du comportement de chaque modèle met en lumière des aspects conceptuels majeurs de la science des données appliquée à la cybersécurité.

Pour le modèle Random Forest, le score de 80.56% démontre l'aptitude des arbres de décision à segmenter l'espace des données via des fenêtres de seuils. En analysant la distribution, le modèle identifie

que les paquets Telnet transmettent majoritairement des frappes de caractères individuelles, générant une profusion de petits paquets de tailles fixes très courtes (par exemple, des longueurs inférieures à 80 octets). À l'inverse, les sessions SSH génèrent des paquets de taille plus importante et variable en raison du bloc de chiffrement et des données d'authentification asymétriques complexes. Cependant, la présence du trafic TCP de contrôle pur (les paquets de synchronisation SYN, de validation ACK sans données applicatives, ou de fin de session FIN) introduit un bruit de fond important : ces paquets de signalisation possèdent des tailles identiques (souvent 66 ou 74 octets) qu'ils appartiennent à une session Telnet ou SSH, limitant ainsi la précision absolue à environ 80%.

Pour le modèle SVM, l'approche adoptée a consisté à appliquer un filtrage préalable via l'instruction 'df = data[data["Protocol"] != "TCP"]'. Ce nettoyage de données supprime le bruit lié aux paquets de signalisation pour se focaliser exclusivement sur les paquets portant une charge utile applicative (Payload). En éliminant cette ambiguïté, le classificateur linéaire SVM parvient à établir un seuil de séparation (une frontière de décision unidimensionnelle) plus net, ce qui propulse sa précision à 81.25%. Cela démontre la pertinence d'intégrer une expertise métier en réseau (savoir isoler le trafic applicatif du trafic de contrôle) pour optimiser les performances des modèles mathématiques de Machine Learning.

## 4.3 Limites Globale du Projet et Perspectives

Bien que l'expérimentation valide les concepts fondamentaux, plusieurs limitations techniques méritent d'être soulignées afin d'ouvrir des perspectives d'amélioration :

• Unidimensionnalité des caractéristiques : L'utilisation exclusive de la caractéristique Length restreint la capacité de généralisation du modèle. Face à des flux réseau réels complexes ou à l'utilisation de techniques d'obfuscation de trafic (comme le rajout de paquets de bourrage ou padding), le modèle pourrait faillir. Il serait hautement préférable d'introduire des caractéristiques complémentaires telles que la variance des intervalles de temps inter-paquets (Delta Time), la direction du flux (Client vers Serveur vs Serveur vers Client), ou des analyses d'entropie de la charge utile pour détecter le chiffrement.

• Environnement de test restreint : Les captures ont été réalisées en boucle locale (localhost). En situation réelle de production, le trafic réseau intègre des contraintes supplémentaires comme le partitionnement des paquets, la retransmission due aux pertes de paquets, et le multiplexage avec de nombreux autres protocoles (HTTP, DNS, TLS).

• Volumétrie des données : Le dataset extrait comporte un nombre de lignes restreint (une centaine de paquets). Pour déployer un modèle robuste en Intelligence Artificielle, des captures s'étalant sur plusieurs heures avec une diversité d'utilisateurs et de commandes exécutées sont indispensables pour construire un échantillon d'apprentissage statistiquement représentatif.

# Conclusion Générale

En conclusion, ce mini-projet académique a permis d'explorer de manière concrète et empirique les fondements de la sécurité des réseaux et l'apport indispensable des mécanismes cryptographiques. À travers la mise en œuvre pratique de Telnet et de SSH sur une plateforme Ubuntu, nous avons matérialisé la transition historique et technique entre des protocoles archaïques transmettant des données sensibles en clair et des solutions modernes protégeant la confidentialité et l'intégrité des flux par chiffrement.

L'utilisation de l'analyseur Wireshark a constitué un jalon pédagogique majeur, nous permettant d'observer directement la vulnérabilité de Telnet (où la capture du mot de passe s'effectue de manière triviale) et la robustesse de SSH, dont le flux se résume à une suite de caractères chiffrés mathématiquement indéchiffrables pour un observateur externe indiscret.

Enfin, l'ouverture vers le Machine Learning a mis en évidence la convergence naturelle entre la sécurité réseau et l'Intelligence Artificielle. En entraînant des modèles Random Forest et SVM avec une précision supérieure à 80% sur la seule base de la longueur des paquets, ce projet démontre qu'il est possible de caractériser et de classifier automatiquement la nature cryptographique d'un flux réseau sans avoir besoin d'en inspecter le contenu privatif. Ce travail pose ainsi les bases méthodologiques des systèmes modernes de détection d'intrusions (IDS) et d'analyse comportementale du trafic, des compétences clés pour notre future insertion dans le domaine de l'Intelligence Artificielle appliquée à la cybersécurité.